TOAP: The Token-Optimized Agent Protocol

A Reference-Minimized, Two-Plane Architecture for Inter-Agent Messaging, with an Honest Token Accounting and a Multi-Model Pilot

Trilochan Sharma
Independent Researcher
parnishklpo@gmail.com

June 2026

Code and data: https://github.com/parnish007/TOAP — Code: MIT — Paper: CC BY 4.0

Abstract

Multi-agent LLM pipelines commonly forward a growing transcript — the source document plus every prior agent's output — into each downstream prompt, so token cost grows with depth. A natural remedy is to pass a *reference* to shared content instead of re-sending it. We report a **preliminary pilot** that measures this remedy honestly rather than advocating it. Three findings stand out. **(1)** In a controlled experiment (n=33 real Claude generations across Haiku/Sonnet/Opus, three context strategies, accuracy scored by a *deterministic, model-independent* rubric checker), reference-minimization (TOAP) and a competent summarizing orchestrator *both* cut downstream tokens at **full accuracy parity (all 33 samples 4/4)**. **(2)** Crucially, the *summarizing baseline saves more tokens than TOAP* ($\sim$2.5$\times$ vs. $\sim$1.9$\times$ vs. the naive baseline): reference-minimization is *not* a token win over good engineering. TOAP's actual value is therefore *losslessness* (a reference can be re-expanded; a summary cannot), *security* (provenance/identity travel with the reference), and *systematization* at the protocol layer. **(3)** A single-sample accuracy regression we initially observed on Haiku **did not reproduce** at n=5 and is retracted — a concrete illustration of why n=1 agent measurements are unsafe. We also separate wire bytes from model tokens and show symbolic "opcodes" save 50% of bytes but only 0–17% of tokens. We describe the protocol architecture (a two-plane wire format; a reference-materialization spectrum) and a Rust reference implementation, and are explicit throughout about what is *implemented* vs. *proposed*. *Threats to validity are first-class*: a single model family/tokenizer, a same-family judge as a secondary check only, and small n. The mechanism is not novel (blackboard systems, A2A artifacts, ADOL); the contribution is the honest, reproducible accounting.

1 Introduction

In agent pipelines and fan-out workflows, an orchestrator typically forwards a growing transcript into each subsequent agent's prompt. Because LLM cost is paid per prompt token, redundant re-transmission dominates coordination cost. TOAP stores shared content once under an identifier and exchanges references, so each agent retrieves only what it needs.

We are deliberately conservative. The mechanism is old (§2); "10×" claims that count bytes mislead (§3); and the question worth answering is not *whether* referencing saves tokens but *how it compares to a competently engineered baseline*, and whether small models pay an accuracy price.

Our central, somewhat humbling, empirical result is that a good summarizing orchestrator saves *more* tokens than reference-minimization, at equal accuracy — so TOAP must justify itself on losslessness, security, and systematization, not on raw token savings.

What is measured vs. proposed. The empirical results (§3, §8) use the implemented V1 text protocol and real model generations. The architecture (§4) describes the full design; **§4.5 states precisely which parts are implemented today and which are design proposals not yet evaluated** (notably the V2 binary plane and the KV-bridge materialization).

Contributions (all framed as a preliminary pilot).

1. A two-plane protocol architecture and a reference-materialization spectrum (§4), with a Rust reference implementation (§5).
2. An honest token accounting separating bytes, tokens, and compute, showing symbolic opcodes are a minor lever (§3).
3. A controlled, n=33, bias-free-scored comparison of naive / summarizing / reference strategies, with a retracted single-sample regression (§8).
4. A direct answer to "why a protocol rather than a better orchestrator?" (§6).

2 Background and Related Work

Shared-store coordination. Agents coordinating through a common store rather than passing full messages is the *blackboard* model (Hearsay-II [1]); database normalization and pointers are the same principle. TOAP's context store is a modern restatement, and we claim no novelty for it. **Agent protocols.** MCP [2] (agent–tool) and A2A [3] (agent–agent) are the dominant standards; A2A already references shared content by ID via *artifacts*. Token bloat is recognized: the ADOL Internet-Draft [4] and MCP SEP-1576 target token-efficient messaging (Table 1). **Caching.** Provider prompt caching [5] realizes "send once, reference implicitly" within one provider/session; cross-agent referencing is complementary. **Symbolic comms** [6, 7] cut tokens but risk comprehension; we measure the opcode component at $\leq$17%. **Security.** Provenance tracking follows CaMeL [8]. **KV reuse** [9] is crowded and is treated as an optional, constraint-guarded mode.

Protocol	Layer	Wire format	Token efficiency a goal?
MCP (Anthropic)	agent–tool	JSON-RPC 2.0 / HTTP	No
A2A (Google/LF)	agent–agent	JSON-RPC/HTTP (+gRPC/REST)	No
ADOL (IETF draft)	layer above MCP/A2A	JSON `$ref` dedup	Yes (content selection)
TOAP (this work)	agent–agent	two-plane text/binary	Yes (encoding, token-aware)

Table 1: Inter-agent protocol landscape (2026). TOAP is adjacent to ADOL/SEP-1576, not ahead of them.

3 Bytes Are Not Tokens

A common error is to minimize *bytes* and report it as token savings. BPE tokenizers pre-tokenize on punctuation before merging, so dense punctuation each costs a whole token. Measured with `tiktoken` (cl100k): `SUM(CTX:42)?max_words=150` is 25 bytes / 10 tokens vs. the NL "Please summarize document 42 in at most 150 words." at 50 bytes / 12 tokens — 50% fewer bytes but only

17% fewer tokens (Figure 1); richer payloads save *less*. The design consequence: **opcode terseness is a minor lever**; the real question is whether to transmit the *content* at all. We therefore separate three cost planes — wire bytes, model-input tokens, prefill compute — and measure each independently.

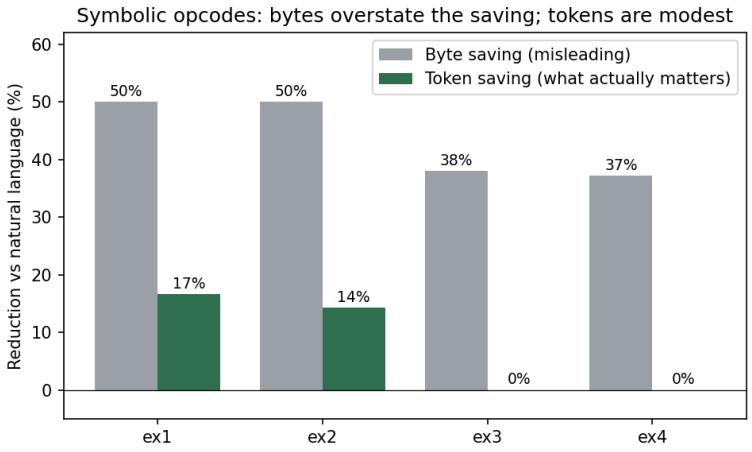

Figure 1: Byte savings overstate token savings for symbolic opcodes (tiktoken cl100k).

4 TOAP Architecture

4.1 System overview

TOAP is a layered, broker-mediated protocol (Figure 2). A message travels *down* the stack on the way out and *up* the stack on the way in; each layer has a single, narrow responsibility. The defining structural choice is that **agents never communicate directly** — every message is mediated by a broker that owns identity, access control, and routing. This is what makes source identity un-spoofable and lets security be enforced in exactly one place.

We describe each layer in turn (§4.1.1–§4.1.6), then the two cross-cutting ideas that distinguish TOAP: the two-plane wire format (§4.2) and the reference materialization spectrum (§4.4).

4.1.1 Agents and the client SDK

Agents (LLMs, tools, or rule-based workers) hold no protocol state of their own; they use a thin async client SDK that performs the SYN/ACK handshake binding identity to the connection, correlates each reply to its request by `msg_id` (so concurrent in-flight requests cannot be confused), and delivers inbound *events* on a channel separate from inbound *requests*. The agent surface is deliberately small: `set_context`, `get_context`, `send_request`, `patch`, `subscribe`.

4.1.2 Protocol layer

The protocol layer turns a message into bytes and back, validating grammar on the way in. V1 is a text frame `TYPE|MSG_ID|TARGET|PAYLOAD` with a function-style payload `OP(args)?k=v&...`; the function form is used precisely because the earlier colon form (`SUM:CTX:42`) is ambiguous when an argument (`CTX:42`) itself contains a colon. The parser is *strict about protocol structure* (delimiter count, payload grammar, context-reference validity) but *never rejects ordinary content*

for resembling an attack — SQL, HTML, code, and even "ignore previous instructions" are valid data. V2 adds a compact binary control header (§4.2) that round-trips the identical message model.

4.1.3 Security filter

Before routing, every inbound message passes four checks, in order: (i) *identity* — derived from the authenticated session, never read from the wire; (ii) *ACL* — per-context read/write/delete; (iii) *capability* — does this content's provenance permit this operation (§4.3); and (iv) *rate/replay* — a per-agent token bucket and per-session duplicate-`msg_id` rejection. A failure at any stage short-circuits to a typed error (`NOPERM`/`RATE`/`REPLAY`) and the message is never routed.

4.1.4 Broker / router

The broker binds one `agent_id` per connection at SYN, routes `REQ` to the target agent, and correlates the returning `RES`/`ERR` to the original requester by `msg_id` — so a responder addresses its reply to an id, never to a (spoofable) identity, and never learns who asked. A `PATCH` (`DLT`) to a subscribed context fans out an `EVT` to every subscriber. The broker is, by design, the sole chokepoint; we treat its availability as an explicit threat-model assumption (§9).

4.1.5 Shared context store

Content is stored once under a numeric id and addressed as `CTX:N`. Each entry carries not just bytes but the metadata that makes a reference safe and useful: broker-derived owner, ACL, capability provenance, TTL (with lazy and swept garbage collection), a monotonic version with an append-only field-level delta log, and a subscriber set. This is what lets a reference be *lossless* (the content is recoverable), *secure* (provenance travels with it), and *collaborative* (versioned deltas + events). The current backend is in-memory and single-node; durable/distributed backends are future work.

4.1.6 Transport

The lowest layer frames messages as length-prefixed records over TCP (Tokio). WebSocket and gRPC transports are planned; nothing above the transport layer depends on the choice.

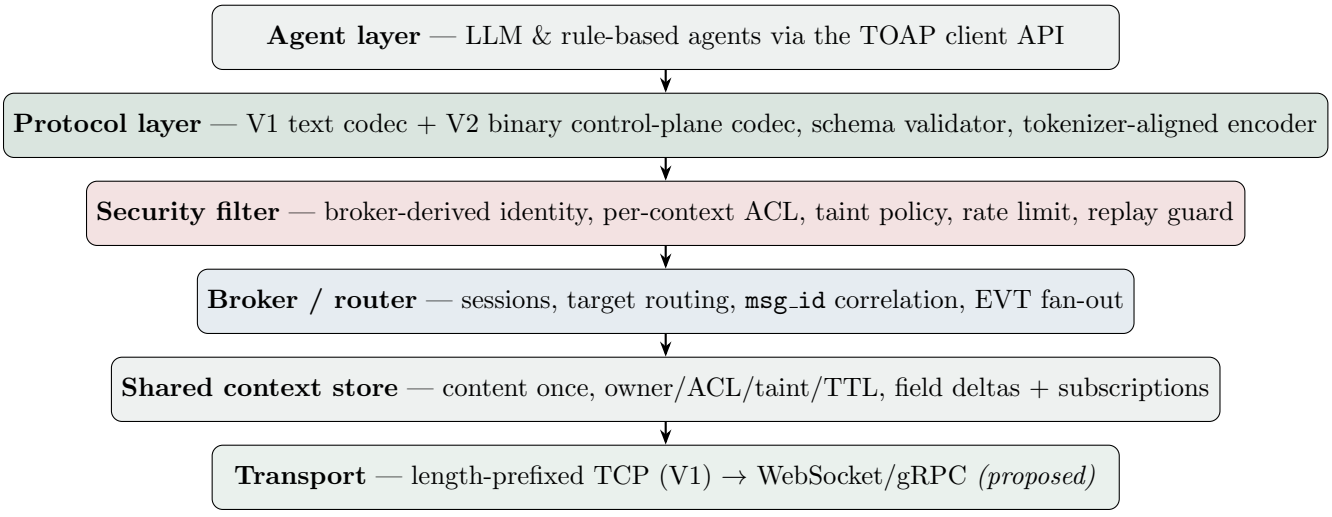

Figure 2: TOAP layered architecture. Identity/trust are broker-owned, never claimed on the wire.

4.2 Two-plane wire format

A single message is read by two parties with opposite cost functions. The *broker* needs ids, session, ACL, capability tags, and nonces — metadata whose cost is *bytes* (bandwidth, parse latency, framing) and which the LLM should never see. The *LLM* needs the operation and its content — text whose cost is *tokens* and which the broker should never tokenize. TOAP encodes these as two planes: a compact binary *control plane* optimized for bytes, and a tokenizer-aligned *semantic plane* optimized for tokens. Because each plane is encoded for its single reader, neither is dominated — which is precisely why "bytes vs. tokens" (§3) stops being a confusion: they are different planes with different objectives.

4.3 Capability-lattice provenance

A boolean "tainted" flag cannot express *what* untrusted content may do. TOAP attaches a provenance to each context: an `Origin` (INTERNAL/EXTERNAL/USER) and the set of `Capability` values that origin permits (read, summarize, transform, classify, execute, email, pay, delete). Trusted internal content permits all; external content permits read-only flows (read/summarize/transform/classify) but no side effects; user-originated content is most restricted. The broker maps each opcode to a capability (`Capability::for_op`) and refuses any operation the provenance forbids — e.g. an `EXEC/PAY/DELETE` driven by user-origin content returns `NOPERM reason=capability_denied`. Because the tag travels with the reference, taint is preserved across agent-to-agent hops, structurally containing prompt-injection propagation rather than relying on fragile content pattern-matching.

4.4 Reference materialization spectrum

"Send the text," "send a context id," and "send a KV-cache pointer" are not three features but three *materializations of one logical reference*, chosen per call by a cost model and degrading gracefully when constraints fail: KV_BRIDGE → CTX_REF → INLINE. INLINE embeds the bytes (best for tiny/one-shot content); CTX_REF sends the id and the receiver fetches from the store (the dedup win for reused content); KV_BRIDGE points at a reusable KV cache so the receiver skips prefill (valid only for the same model and tokenizer). The same decision — materialize now vs. reference vs. reuse computation — spans both the token plane (resend text vs. send id) and the tensor plane (re-prefill vs. load KV); treating them as one policy is the architecture's most novel point. Figure 3 shows the token-flow contrast the experiment measures: naive prompts grow with pipeline depth while references stay flat, because the store holds content once and only ids travel.

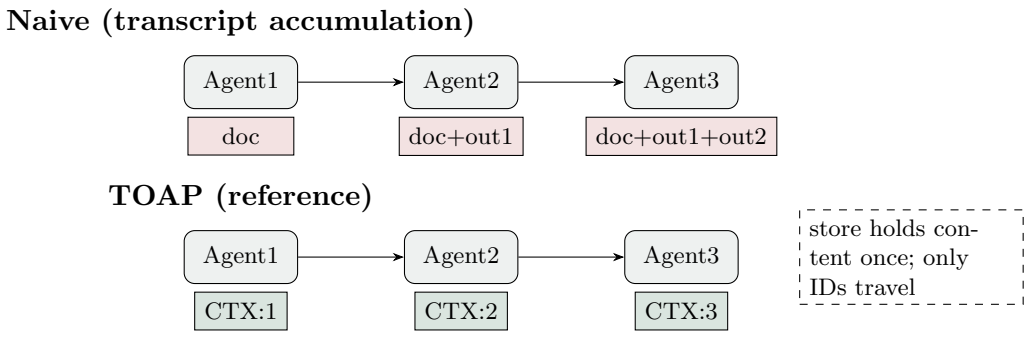

Figure 3: Token-flow contrast: naive prompts grow with depth; references stay flat.

4.5 Implemented vs. proposed

Implemented today (28 passing unit + integration tests): the V1 text protocol and parser; the **V2 binary control plane** (compact binary control header + text semantic payload — the two-plane design, with round-trip tests); the shared context store with per-context ACL, TTL, field-level deltas, and subscriptions; a **capability-lattice** provenance model (Origin ∈ {Internal, External, User}, each with an allowed-capability set) the broker enforces structurally (user-/external-origin content cannot flow into EXEC/PAY/DELETE); an **in-band cache coordinate** with a stable-prefix ‖ volatile-suffix layout helper; the **materialization policy with graceful fallback** (KV_BRIDGE → CTX_REF → INLINE) behind a `KvTransport` trait; broker-derived identity, token-bucket rate limiting, per-session replay rejection, `msg_id` correlation; and an MCP frontend.

Still requires a model runtime (interface implemented, tensor transfer not): the KV-cache *transport* behind `KvTransport` — with no runtime the policy correctly degrades to CTX_REF (the tested default). Cross-vendor evaluation also remains future work.

5 Implementation

TOAP is a Rust workspace. `toap-core` defines the message model, a strict recursive-descent parser, and the encoder; it deliberately rejects malformed *protocol* syntax but never rejects ordinary content for "looking dangerous" (SQL/HTML/prompt-like text are data). `toap-context` implements the in-memory store: numeric context IDs, an ACL grammar (`agent:rwd`), a *capability-lattice* provenance model (`Origin` × allowed `Capability` set, with `Capability::for_op` mapping opcodes to capabilities), TTL with lazy + sweep GC, a bounded per-context delta log, the `Reference` materialization enum with a `materialize_with_fallback` policy behind a `KvTransport` trait, and a `cache_layout` helper for stable-prefix ordering. `toap-core` also provides a V2 binary control-plane codec (`encode_v2`/`decode_v2`) that round-trips the V1 message model. `toap-security` provides a token-bucket rate limiter and a per-session replay guard; the broker enforces the capability lattice structurally, refusing e.g. an `EXEC/PAY/DELETE` against user- or external-origin context. `toap-broker` is an asynchronous (Tokio) TCP server with length-prefixed framing; identity is bound to the connection at SYN, and a responder addresses replies by the original `msg_id` so it never learns the requester (preventing source spoofing). `toap-client` is an async SDK; `toap-mcp` is the MCP frontend.

Testing is unit *and* integration: parser round-trip/validation and store/ACL/TTL/delta unit tests, plus end-to-end integration tests in which **real client processes connect to the broker over TCP** and exchange requests — the broker performs actual routing and store operations (not mocked). Security integration tests assert that an unauthorized read is denied, a restricted op on tainted context is refused, and a subscriber receives a delta event. A live multi-process demo (broker + summarizer + classifier + orchestrator) exercises fan-out and merge. Coordination-frame sizes reported below are measured on the actual encoder output.

6 Why a Protocol, Not Just a Better Orchestrator?

The sharpest objection is: "just write an orchestrator that summarizes before forwarding." Our experiment confirms a summarizing orchestrator is in fact *more* token-efficient than referencing (§8). So the case for a protocol does not rest on token savings. It rests on three properties a prompt-engineering pattern does not provide: **(i) Losslessness** — a reference can be re-expanded to the full content on demand; a summary discards information irreversibly, which is safe only until a downstream task needs a dropped detail. **(ii) Security travels with the reference** —

provenance/taint and broker-derived identity are enforced structurally, so a compromised agent cannot launder untrusted content or spoof a source; a summarizing prompt template offers none of this. **(iii) Systematization** — the behavior is uniform across heterogeneous agents and languages and composes with deduplication (send-once/refer-many) and provider caching, rather than being re-implemented per application. TOAP should be adopted for these reasons, or not at all.

7 Experimental Method

Instrument. Each agent call is an isolated Claude subagent at an explicit model (Haiku/Sonnet/Opus), tool use disabled; outputs are genuine generations. **Controlled design.** To get clean variance we fix one canonical upstream (one document, one extracted-facts text, and one analysis containing all four rubric themes) so the only variables are {model, context strategy, sample}. Three strategies for the final *writer* stage: *naive* (document + facts + analysis), *summary* (a short lossy summary of the analysis, i.e. a competent summarizing orchestrator), and *TOAP* (the analysis only, i.e. the distilled slice passed by reference). We draw n=5 samples per cell for Haiku and n=3 for Sonnet/Opus (n=33 total). **Accuracy (bias-free).** The primary accuracy measure is a *deterministic* rubric checker: each of four required actions is detected by keyword sets, with no model in the loop, so it is immune to same-family judge bias and exactly reproducible. (A model judge was used only in an earlier single-sample run and is not relied upon here.) **Tokens.** Counted with `tiktoken` (cl100k) on the exact prompts and verbatim outputs. **Reproducibility.** All prompts, verbatim outputs, and the scorer are committed.

Reviewer-driven changes from the first draft. (1) n=1 replaced by n=33 with ranges; (2) a *strong* summarizing baseline added alongside the naive one; (3) a deterministic, model-independent scorer replaces reliance on a same-family judge; (4) the architecture now flags implemented vs. proposed; (5) all empirical claims framed as a preliminary pilot.

8 Results

Accuracy parity; the regression retracted. All 33 samples scored 4/4 on the deterministic rubric (Figure 5); coverage variance was zero. A 4/4→3/4 Haiku regression seen in an earlier *single-sample* run **did not reproduce** at n=5 with a fixed upstream; we attribute it to single-sample noise and/or propagation of a weaker upstream analysis, and retract it.

Tokens: the summarizing baseline beats TOAP. Table 2 and Figure 4 give total writer tokens. Versus the naive baseline, TOAP reduces tokens by **1.88–1.93**× (mean ∼1.91×) while the summarizing baseline reduces them by **2.39–2.62**× (mean ∼2.54×), *both at 4/4 accuracy*. Reference-passing is therefore *not* the most token-efficient strategy; a competent summarizer is. This is the central, honest result.

Where referencing does win: multi-hop and coordination. The writer experiment is a single hop. In a multi-hop pipeline, referencing avoids re-summarizing/re-sending at *every* hop, and the store's send-once/refer-many compounds; an exploratory three-stage run showed 1.65–1.82× downstream reductions. Separately, TOAP coordination frames are ∼17× smaller in tokens than JSON-RPC equivalents (model-independent, exactly reproducible). These are the regimes where a reference protocol, rather than per-hop summarization, is the natural fit — and where losslessness matters most.

Model	Strategy	n	Accuracy (/4)	Total tokens (mean [min–max])	Reduction vs. naive
Haiku	naive	5	4.0	505 [491–520]	—
Haiku	summary	5	4.0	211 [193–219]	2.39×
Haiku	TOAP	5	4.0	268 [258–279]	1.88×
Sonnet	naive	3	4.0	512 [499–523]	—
Sonnet	summary	3	4.0	195 [190–204]	2.62×
Sonnet	TOAP	3	4.0	266 [265–267]	1.93×
Opus	naive	3	4.0	504 [487–519]	—
Opus	summary	3	4.0	194 [192–196]	2.60×
Opus	TOAP	3	4.0	263 [253–271]	1.92×

Table 2: Controlled writer experiment (n=33). Accuracy by deterministic rubric (bias-free). Tokens via tiktoken. TOAP < naive, but summary < TOAP.

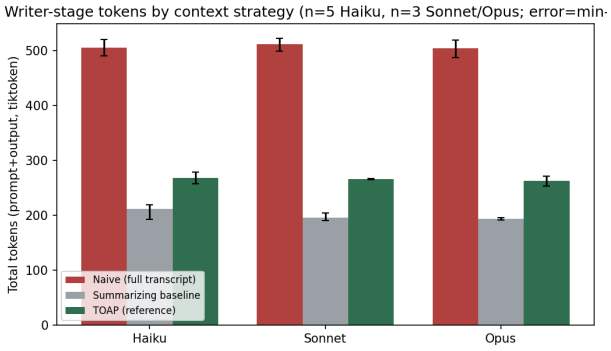

Figure 4: Writer tokens by strategy (error = min–max). Summary < TOAP < naive.

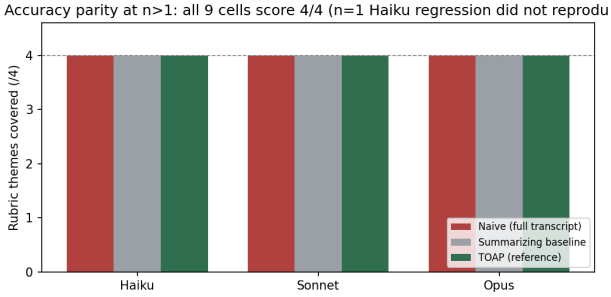

Figure 5: Accuracy parity: all nine cells 4/4 (the n=1 regression did not reproduce).

9 Discussion and Limitations

The honest synthesis: *naive transcript accumulation is wasteful; both summarization and referencing fix it at equal accuracy here; summarization is cheaper on tokens but lossy; referencing is slightly costlier but lossless, secure, and systematic* (§6). The retracted Haiku regression is itself a result: **single-call agent measurements are unsafe**, and apparent small-model effects must be checked with replication.

We state threats plainly and treat them as first-class. **Single model family / one tokenizer:** Haiku/Sonnet/Opus differ by scale, not architecture; token claims are tokenizer-dependent, so cross-vendor (GPT/Gemini/Llama) replication is required and untested. **Small n and a single task:** n=33 on one incident scenario with a fixed upstream; broader datasets are needed. **Saturated accuracy:** the four-theme task was easy enough that all strategies reached 4/4, so it does not stress the losslessness advantage — a harder task that needs a detail a summary would drop is exactly the missing experiment. **Scorer:** keyword coverage is bias-free but coarse; it can miss paraphrase. **Judge:** any model judge here is same-family and is used only as a discarded secondary check. **Implementation gap:** the V2 binary control plane, capability lattice, cache-coordinate layout, and materialization-with-fallback are now implemented and unit-tested; only the actual KV-cache *tensor transport* still requires a model runtime (without one the policy degrades to CTX_REF) and is not benchmarked here.

On novelty. We claim none for the mechanism. The contribution is the controlled, bias-free, reproducible accounting — including the finding that a summarizing baseline beats referencing on

tokens — and an architecture that makes the byte/token planes and the materialization choice explicit.

10 Conclusion

Reference-minimized inter-agent messaging removes the waste of transcript accumulation at no measured accuracy cost, but it is *not* the cheapest strategy: a competent summarizing orchestrator saves more tokens. TOAP earns its place only through losslessness, in-band security, and systematization, and in multi-hop/coordination regimes where per-hop summarization is awkward and lossiness is risky. We release all artifacts and call for cross-vendor, large-n, harder-task replication — especially a task that stresses the lossless-vs-lossy distinction — before any general claim.

References

[1] L. Erman, F. Hayes-Roth, V. Lesser, D. Reddy. The Hearsay-II Speech-Understanding System (blackboard model). *ACM Computing Surveys* 12(2), 1980.

[2] Anthropic. Model Context Protocol specification, 2024–2026. https://modelcontextprotocol.io.

[3] Google / Linux Foundation. Agent2Agent (A2A) Protocol v0.3, 2025–2026. https://a2a-protocol.org.

[4] Z. Chang, J. Li, Z. Cao. A Token-Efficient Data Layer for Agents (ADOL). IETF Internet-Draft `draft-chang-agent-token-efficient-01`, Dec. 2025.

[5] Anthropic, OpenAI, Google. Prompt caching documentation, 2024–2026.

[6] Z. Wang et al. Talk Structurally, Act Hierarchically (TalkHier). arXiv:2502.11098, 2025.

[7] CodeAgents: pseudocode-codified multi-agent messages. arXiv:2507.03254, 2025.

[8] Google DeepMind. Defeating Prompt Injections by Design (CaMeL). arXiv:2503.18813, 2025.

[9] KVCOMM: Online Cross-context KV-cache Communication. arXiv:2510.12872, 2025.

A Reproducibility

Artifacts: `benchmark/ab_study/` (harness, scenarios, the n=33 run records `runs/writer_runs.json`, deterministic scorer `writer_experiment.py`, analysis `compute_writer.py`); `paper/figures/` (figure scripts); `paper/METHODS.md` and `paper/REVIEW_RESPONSE.md` (method log and the reviewer-fix trail). The Rust implementation is in `crates/` (28 unit + integration tests, incl. real-TCP end-to-end, V2 binary round-trip, capability-lattice flow, cache layout, and KV-bridge fallback). Token reductions recompute exactly from the committed records via `compute_writer.py`.